
MARGENT: Measuring the Marginal Value of Delegation in Agentic Systems

Anonymous Author(s)

Affiliation

Address

email

Abstract

Current agentic systems increasingly rely on multiple specialized agents to improve performance but each additional interaction requires more model computation and introduces new failure paths that may leave the solution unchanged or even corrupt a correct answer. Many existing systems follow fixed interaction patterns or, when selective, route from query-level features or model confidence; these signals do not directly compare the outcomes of committing and delegating from the same reasoning state. This raises a fundamental question for agentic systems: *When is additional intelligence worth it?* We study this question in a concrete multi-agent setting, where a manager must either commit to its current candidate solution or delegate to another sub-agent. We introduce MARGENT, a trainable framework that branches from the same manager-generated reasoning state into immediate commitment and delegation to each available sub-agent. By comparing their final-answer correctness, MARGENT measures the marginal value of each delegation and distills the shortest successful branches into supervised trajectories for policy learning. Across four reasoning benchmarks, the best single delegation recovers 91.7% of the total accuracy gain available under the measured oracle on average. On four collection-disjoint evaluation sets, the final system exceeds the same trained manager’s pre-delegation accuracy by 17.1 percentage points while making 0.53 sub-agent calls per example. On development diagnostics, with roughly one selected call instead of three, it also matches forced-all delegation on MMLU-Pro (71.5%) and exceeds it on AQuA-RAT (81.1% vs. 75.2%). These results show that effective agentic systems depend not on invoking more agents, but on allocating additional intelligence according to its marginal value at the current reasoning state.

1 Introduction

Agentic systems extend the capabilities of large language models by coordinating specialized components rather than asking a single model to solve each problem from beginning to end [Wu et al., 2024, Hong et al., 2024]. In multi-agent systems, sub-agents can propose solutions, exchange evidence, and critique one another, improving reasoning and factual accuracy on some tasks [Du et al., 2024, Chan et al., 2024]. These gains, however, are not free. Every delegation requires additional model computation and creates a new failure path: a sub-agent may repeat existing work, introduce inconsistent evidence, or replace a correct solution with an incorrect one [Cemri et al., 2025, Eo et al., 2025].

Many multi-agent systems follow fixed interaction patterns. Selective methods instead adapt team structure, prune communication, route queries, or trigger debate from model uncertainty [Liu et al., 2024, Zhang et al., 2025, Yue et al., 2025, Eo et al., 2025]. Although these methods make collaboration selective, many decisions are based on query-level features, team structure, or model confidence. Such signals do not directly determine whether a specific sub-agent can improve the candidate solution

38 already under consideration. A low-confidence solution may not be repairable by any available
 39 sub-agent, while a high-confidence solution may contain an error that another sub-agent can detect.
 40 The same delegation can therefore rescue one solution, leave another unchanged, and corrupt a third.
 41 This raises a fundamental question for agentic systems: *When is additional intelligence worth it?*

42 We study this question in a concrete multi-agent setting. At each decision point, a trainable manager
 43 first generates the candidate solution it would submit if it stopped. The manager policy then selects
 44 its own next action: commit to that solution or delegate to one of several frozen sub-agents. After a
 45 delegation, the manager updates its candidate solution and decides again. We introduce MARGENT,
 46 a trainable framework that measures delegation from the same manager-generated reasoning state.
 47 During data collection, MARGENT forces immediate commitment and each available delegation as
 48 separate branches and compares their final-answer correctness. This paired comparison identifies
 49 whether a call rescues, leaves unchanged, or corrupts the current solution while holding the query,
 50 history, and candidate solution fixed.

51 Learning from these measurements introduces a separate challenge. A terminal correctness reward
 52 cannot distinguish immediate commitment from an unnecessary delegation when both paths are
 53 correct, while a call penalty introduces a task-dependent trade-off between accuracy and resource
 54 use. MARGENT instead uses a correctness-first, shortest-successful-path rule: it teaches the manager
 55 to commit when the current candidate solution is correct and otherwise distills a shortest measured
 56 branch that reaches a correct answer. Only the manager is fine-tuned; the sub-agents remain frozen.
 57 The resulting supervised trajectories train the manager when to commit and when to delegate without
 58 assigning a fixed value to any sub-agent.

59 Across four reasoning benchmarks, the best single delegation recovers 91.7% of the accuracy gain
 60 between immediate commitment and the deepest measured oracle on average. On four collection-
 61 disjoint evaluation sets, the final system exceeds the same trained manager’s pre-delegation accuracy
 62 by 17.1 percentage points while making 0.53 sub-agent calls per example. On development diagnos-
 63 tics, with roughly one selected call rather than three, it matches forced-all delegation on MMLU-Pro
 64 and exceeds it on AQuA-RAT. Removing the candidate solution passed to the VERIFIER further
 65 reduces AQuA-RAT accuracy from 81.1% to 74.0%, showing that the current candidate can materially
 66 affect the usefulness of a delegated verification call.

67 **Contributions.** Our work makes three main contributions:

- 68 • **State-conditioned delegation value.** We define the marginal value of a sub-agent call by
 69 comparing delegation and commitment from the same manager-generated reasoning state.
- 70 • **A trainable delegate-or-commit framework.** We introduce MARGENT, which collects
 71 paired outcomes through same-state branching and distills shortest successful branches into
 72 supervised trajectories for policy learning.
- 73 • **Evidence for selective allocation.** Across four benchmarks, held-out evaluations and
 74 controlled diagnostics show that the learned manager recovers much of the measured multi-
 75 agent gain with selective calls, while targeted interventions probe when and why calls
 76 help.

77 2 Related work

78 **Selective collaboration.** Multi-agent debate and role-based workflows often use fixed interaction
 79 patterns [Du et al., 2024, Hong et al., 2024]. DyLAN and AgentPrune adapt team structure or remove
 80 redundant communication [Liu et al., 2024, Zhang et al., 2025]; MasRouter selects collaboration
 81 modes and model configurations [Yue et al., 2025]; and Debate Only When Necessary gates interaction
 82 using confidence [Eo et al., 2025]. More broadly, intelligent delegation has been framed around
 83 capability, cost, monitoring, trust, and authority [Tomašev et al., 2026]. MARGENT instead asks
 84 whether a named sub-agent improves the manager’s current draft, a state that changes after every
 85 interaction.

86 **Routing and learned orchestration.** Learning to defer, model routing, and cascades choose an
 87 expert or model under quality–cost constraints [Mozannar and Sontag, 2020, Verma et al., 2023,
 88 Chen et al., 2023, Ong et al., 2024]. In our setting, sub-agents provide signals and the manager

89 still produces the final answer. Recent systems train multi-agent components from outcome-graded
 90 trajectories or optimize planners inside live loops [Motwani et al., 2025, Park et al., 2025, Li et al.,
 91 2026]. Our contribution is the same-state action comparison that identifies when a delegation changes
 92 the outcome, followed by policy training from those measured differences.

93 3 MARGENT: Marginal-value-guided delegation

94 MARGENT consists of two stages. First, it evaluates commitment and each available delegation from
 95 the same manager-generated reasoning state, measuring whether a sub-agent changes the correctness
 96 of the current candidate solution. Second, it selects a shortest measured successful trajectory and
 97 distills it into supervision for learning a delegate-or-commit policy. This design separates measuring
 98 whether additional intelligence helps from learning when to use it.

99 3.1 Delegate-or-commit interface

100 For question x , let the state at turn t be $s_t = (x, h_t, \hat{y}_t, U_t)$, containing the interaction history h_t , the
 101 manager’s current candidate solution $\hat{y}_t$, and the set U_t of sub-agents already used. The manager
 102 policy π_θ chooses

$$u_t \in \mathcal{U}(s_t) = \{\text{commit}\} \cup \{\text{delegate}(a) : a \in \mathcal{A} \setminus U_t\}, \quad (1)$$

103 where $\mathcal{A}$ is the set of available sub-agents. A commitment terminates the trajectory and submits $\hat{y}_t$ as
 104 the final answer. A delegation adds the selected sub-agent’s output to h_t , after which the manager
 105 revises its candidate solution and chooses again.

106 The interface permits any finite collection of sub-agents. In our experimental instantiation, $\mathcal{A} =$
 107 $\{\text{EXTRACTOR}, \text{REASONER}, \text{VERIFIER}\}$. The Extractor identifies decision-relevant facts and their
 108 polarity, the Reasoner decomposes the problem and identifies missing reasoning steps, and the Verifier
 109 audits the current candidate solution. The sub-agents are frozen and return structured decision signals
 110 rather than a choice label or final answer; the manager retains control of both solution revision and
 111 final commitment. Each sub-agent can be called at most once, so a trajectory in our setting contains
 112 at most three delegations.

113 We use greedy decoding for sub-agent generation. The Extractor and Reasoner are conditioned on
 114 the question and its context, while the Verifier additionally receives the manager’s current candidate
 115 solution. Cache keys therefore include the complete input to each sub-agent, including the candidate
 116 solution for the Verifier. Teacher-synthesized sub-agent data are filtered using schema-validation and
 117 explicit answer-leakage checks. The appendix summarizes the retained reproducibility metadata;
 118 fields absent from the exported record are not reconstructed.

119 3.2 Interventional marginal-value measurement

120 Let $C(s_t, \text{commit}) \in \{0, 1\}$ indicate whether committing the current candidate solution is correct.
 121 For a one-step delegation probe, we force the manager to delegate to sub-agent a , expose its response
 122 to the manager, and then force the manager to revise and commit without another delegation. Denote
 123 the correctness of this branch by $C(s_t, \text{delegate}(a))$. The marginal value of delegating to a at state
 124 s_t is

$$\Delta(s_t, a) = C(s_t, \text{delegate}(a)) - C(s_t, \text{commit}) \in \{-1, 0, +1\}. \quad (2)$$

125 A value of $+1$ is a *rescue*, 0 means that the delegation does not change correctness, and -1 is a
 126 *corruption*. Thus, marginal value is specific to both the current reasoning state and the selected sub-
 127 agent, rather than being determined by the query or sub-agent identity alone. The quantity measures
 128 the effect of the complete delegation operation—calling the sub-agent, revising the candidate solution,
 129 and committing—under the fixed revision protocol; it is not an intrinsic measure of sub-agent
 130 capability.

131 For each training question, a temperature-zero manager first produces an initial candidate solution
 132 without delegation. From this shared root state, data collection forces direct commitment and
 133 delegation to each available sub-agent. The gold answer is used only to score the resulting terminal
 134 branches and is never shown to the manager or sub-agents.

135 Search proceeds breadth-first over ordered sequences of distinct sub-agents to a maximum depth
 136 $D \leq 3$. For a directly incorrect candidate solution, the collector evaluates every sequence at the

current depth and stops after the first depth containing at least one successful sequence. Because the entire depth is evaluated before stopping, collection retains all equally short successful sequences. If no sequence succeeds, search continues until depth D . For a directly correct candidate solution, only one-step delegations are evaluated, since direct commitment is already the shortest successful trajectory; these additional branches are retained for corruption analysis.

With three non-repeating sub-agents, the complete ordered search space contains 16 possible sequences, including direct commitment. The breadth-first stopping rule does not require all 16 sequences to be evaluated for every question: it establishes the shortest successful depth after fully evaluating only the necessary search layers.

The root states are generated by the manager, whereas descendant states arise from forced interventions. The alternative actions are therefore compared from a shared manager-generated state, including actions that the current policy might not select naturally. This paired design avoids estimating the value of a sub-agent only on examples to which the current policy already delegates. Equation 2 measures the effect of one delegation followed by immediate commitment. The outcomes of longer delegation sequences are measured separately and should not be conflated with this one-step quantity. In particular, a first action can have $\Delta(s, a) = 0$ under immediate commitment yet still begin a successful two-step continuation; such an action is selected by the sequence criterion below, not by its one-step Δ alone.

3.3 Marginal-value distillation

For each collected root state s , let $\Sigma(s)$ contain the evaluated delegation sequences, including the empty sequence ϵ , and let $c_\sigma \in \{0, 1\}$ denote the final correctness of sequence σ . We define the set of shortest successful sequences as

$$\Sigma^*(s) = \begin{cases} \{\epsilon\}, & c_\epsilon = 1, \\ \arg \min_{\sigma \in \Sigma(s): c_\sigma=1} |\sigma|, & c_\epsilon = 0 \wedge \exists \sigma \in \Sigma(s) : c_\sigma = 1, \\ \emptyset, & \text{otherwise.} \end{cases} \quad (3)$$

Here, $\arg \min$ denotes the set of all minimizing sequences. For every nonempty $\Sigma^*(s)$, we select one trajectory uniformly,

$$\sigma^* \sim \text{Uniform}(\Sigma^*(s)), \quad (4)$$

using a fixed example-specific seed for reproducibility.

A correct candidate solution therefore produces a direct-commit target. An incorrect candidate solution produces one sampled shortest rescue if at least one measured delegation sequence succeeds. States for which no measured sequence succeeds do not produce a routing label: the observations establish neither a successful delegation target nor a successful final solution. This avoids teaching direct commitment merely because every measured alternative also fails.

Each selected trajectory produces a supervised decision at every visited state, followed by a final commitment target. To prevent the training set from being dominated by directly correct examples, we cap the number of commit trajectories at ρ times the number of rescue trajectories; we treat ρ as a data-construction hyperparameter and sweep it in Section 4. Supervised fine-tuning on these selected trajectories produces the learned manager policy π_θ . The loss updates the full manager, so it can change both action selection and post-call revision; candidate-to-final gains are therefore end-to-end rather than routing-only effects. Branches are collected before this update and are not recollected after SFT.

This procedure imposes a lexicographic preference over measured successful trajectories: correctness first, followed by fewer delegations among successful paths. Unlike a scalarized per-call penalty, the trajectory target does not require a numeric exchange rate between correctness and calls; ρ still controls how much direct-commit supervision is retained.

MARGENT: Measuring the Marginal Value of Delegation in Agentic Systems

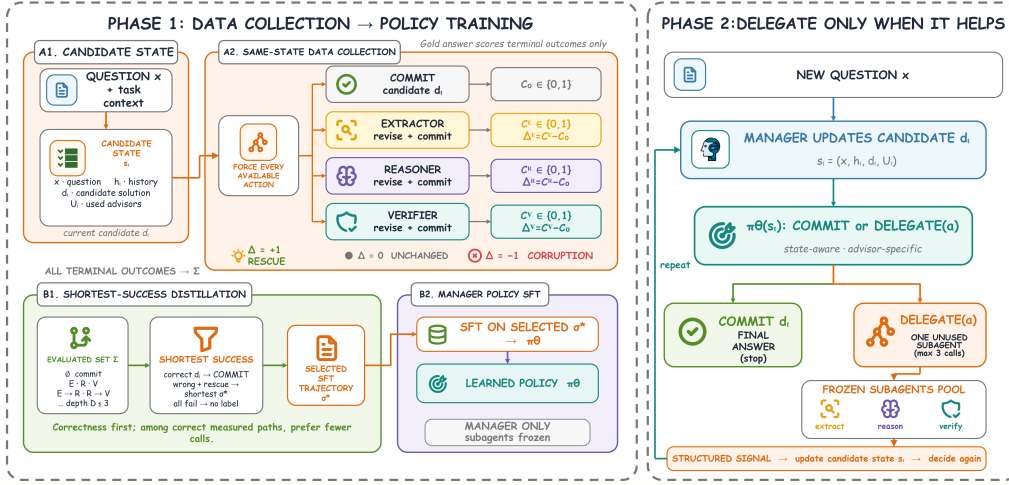


Figure 1: Overview of MARGENT. **Left:** From a shared manager-generated state, MARGENT evaluates commitment and every available delegation, then distills a shortest measured successful trajectory for manager SFT. **Right:** At inference, the learned manager commits or delegates to an unused frozen sub-agent, updates its candidate solution from the structured signal, and decides again.

4 Experiments

4.1 Setup

Benchmarks and splits. We use four multiple-choice reasoning benchmarks: MedQA [Jin et al., 2021], MMLU-Pro [Wang et al., 2024], GPQA [Rein et al., 2024], and AQuA-RAT [Ling et al., 2017], spanning medical, broad-knowledge, scientific, and mathematical questions. Interventional collections contain 400 states for each task except GPQA (200). Policy development uses separate evaluation pools (200 examples for MedQA and MMLU-Pro, 254 for AQuA-RAT, and 100 for GPQA). After fixing depth, ρ , and checkpoint, we run one locked evaluation on MedQA ($n = 500$), MMLU-Pro ($n = 500$), and AQuA-RAT ($n = 254$). GPQA-Diamond ($n = 100$) is disjoint from its collection pool and serves as the fourth collection-disjoint evaluation set.

Models and training. The experiments use a Qwen3.5-9B manager and three frozen role-specific LoRA sub-agents on the same model family. Interventional probes and sub-agent calls use greedy decoding. Two settings are swept, one on the collection side and one on the supervision side. The first is search depth, the maximum number of delegations expanded when the counterfactual tree is collected: we search to depth three for MedQA and AQuA-RAT, and to depth two for the primary MMLU-Pro and GPQA collections. The second is the commit-to-rescue ratio ρ defined in Section 3.3, which controls how many commit states are kept per rescue state when a collection is converted into supervision. At $\rho = 0$ only rescue states are kept, so all retained root supervision begins with delegation; empirically, the recorded ratio-zero policies delegate on nearly or exactly every evaluated question. Larger ρ adds states in which committing to the draft was already correct, and so teaches the policy when not to call. We sweep $\rho \in \{0, 0.5, 1, 2, 3\}$ over the recorded values where available. Selected checkpoints are fixed before the locked evaluations. Appendix A lists the retained split and selection metadata. Optimizer, hardware, token, and latency metadata are not present in the exported experiment record; we therefore make resource claims only in terms of sub-agent calls.

Comparisons and metrics. We compare the learned policy with (i) its own candidate solution before any delegation, (ii) forcing one Verifier call, (iii) forcing all three sub-agents, and (iv) the measured oracle that selects the best evaluated sequence per example. Accuracy is exact-match multiple-choice accuracy. Resource use is the mean number of sub-agent calls. We additionally

report rescue/correction and corruption probabilities from paired outcomes. These baselines probe calling, selection, and sequencing within the same fixed sub-agent pool. Forced-all uses the recorded scaffold order; no separate order sweep is available.

4.2 How much value is available after one delegation?

Table 1: Measured oracle accuracy by maximum delegation depth. Each row is derived from one shared collection, so depth comparisons do not mix runs. Gain@1 is the fraction of the total measured gain obtained by the best one-call branch: $(A_1 - A_0)/(A_D - A_0)$.

Task	n	Commit A_0	Best 1 call A_1	Best measured A_D	Gain@1
MedQA	400	76.00	91.75	92.25	96.9%
AQuA-RAT	400	54.25	85.75	87.25	95.5%
MMLU-Pro	400	53.25	71.75	73.50	91.4%
GPQA	200	43.00	65.00	69.50	83.0%
Macro mean	–	56.63	78.56	80.63	91.7%

Table 1 shows that most recoverable value is concentrated in the first well-chosen delegation. One call recovers more than 90% of the measured gain on three of the four tasks and 91.7% on average. The second call adds only 0.5–4.5 accuracy points; a third call adds exactly zero on the two primary collections evaluated to depth three. GPQA is the exception: its second call adds 4.5 points, and in one depth-two configuration 16% of evaluated examples make more than one call. GPQA is also the task whose commit accuracy sits closest to chance, so the exception fits the same pattern rather than breaking it: the weaker the first answer, the more a second opinion still has to add. Everywhere else the first call is effectively the entire budget. Allocating that call well is therefore the dominant decision in this sub-agent pool. Put differently, a single correctly chosen sub-agent captures most of what the measured tree can deliver: the second call is worth at most 4.5 points, and on the two primary collections evaluated to depth three the third call adds no additional oracle accuracy.

4.3 The value of a call depends on which sub-agent is called

The oracle above chooses among sub-agents after seeing branch outcomes. To separate this selection effect from generic extra computation, we compute $100\bar{\Delta}_a$ for every task–sub-agent pair using the forced one-step branches (Table 2). The range is wide: the same Reasoner has net value +6.8 points on MedQA but −4.3 on AQuA-RAT, so a role that helps on one task actively costs accuracy on another, while the Extractor is close to zero or negative on all four tasks. The Verifier has the largest mean value on all four tasks, but is not interchangeable with the other roles: in the one-step records, 12 MedQA and 9 MMLU-Pro questions are rescued by the Reasoner but not the Verifier, and forcing the strongest average role on every question still spends calls on already-correct solutions.

Table 2: Net marginal value of one forced call, $100\bar{\Delta}_a$, in percentage points. Positive cells contain more rescue than corruption mass; negative cells contain more corruption.

Task	Extractor	Reasoner	Verifier
MedQA	+2.0	+6.8	+10.0
MMLU-Pro	−0.5	+2.3	+15.1
GPQA	+0.5	+4.1	+15.5
AQuA-RAT	−4.5	−4.3	+29.4

These averages explain why task-level routing alone is insufficient. Always choosing the Verifier is a strong task-level rule, but it cannot express the state-level decision to stop, nor can it exploit the minority of solutions for which another role supplies the needed information. In shallow MMLU-Pro and GPQA operating points, the learned policy never selects the near-zero-value Extractor, showing that the distilled action labels recover part of this role ordering without observing gold labels at inference time.

Table 3: Evaluation on sets disjoint from interventional collection. MedQA, MMLU-Pro, and AQuA-RAT use checkpoints fixed before their final evaluation; GPQA-Diamond-100 is collection-disjoint but also serves as its diagnostic pool. “Candidate” is the answer the same manager would submit before its first delegation.

Task	n	Candidate	MARGENT	Gain (pp)	Calls
MedQA	500	73.4	79.8	+6.4	0.460
AQuA-RAT	254	37.4	76.8	+39.4	0.780
MMLU-Pro	500	49.6	60.2	+10.6	0.352
GPQA	100	42.0	54.0	+12.0	0.510
Macro mean	—	50.6	67.7	+17.1	0.526

237 4.4 Learned delegation on collection-disjoint evaluations

238 On the four evaluations in Table 3, MARGENT’s final output is more accurate than the same trained
 239 manager’s pre-delegation candidate on every task. The macro difference is 17.1 percentage points
 240 with 0.53 calls per example. Because SFT updates the full manager, this is an end-to-end difference
 241 inside the same scaffold rather than a routing-only causal effect, and it should not be read as a
 242 gain over an independently prompted single-model baseline. This distinction matters on AQuA-
 243 RAT, where the tool-enabled scaffold’s candidate solution is substantially weaker than a separate
 244 plain-answer prompt.

245 **A strong fixed-role baseline.** Because Table 2 identifies the Verifier as the strongest average role,
 246 always calling it is more informative than comparing only with no calls or all calls. Across four
 247 development sets, low-call MARGENT policies reduce mean usage from 1.00 to 0.565 calls while
 248 giving up 1.6 accuracy points on average. The recorded forced-Verifier implementation does not
 249 pass the same pre-call candidate to the Verifier, so this is a strong operational baseline rather than
 250 a perfectly matched routing ablation. The MedQA entry uses the three-seed mean rather than the
 251 favorable 82.0% @ 0.405 run.

Table 4: Low-call MARGENT policies versus forcing one Verifier call on every example. Results are development diagnostics. MedQA reports mean $\pm$ standard deviation over three additional seeds; other rows are single runs.

Task	MARGENT		Always Verifier		Fewer calls (%)
	Acc.	Calls	Acc.	Calls	
MedQA	80.5	0.517	84.5	1.0	48.3
MMLU-Pro	64.5	0.340	67.0	1.0	66.0
GPQA	54.0	0.510	55.0	1.0	49.0
AQuA-RAT	78.0	0.894	76.8	1.0	10.6
Macro mean	69.3	0.565	70.8	1.0	43.5

252 Table 5 examines the accuracy–call behavior on development pools, where forced baselines were
 253 recorded. At roughly one selected call per example, MARGENT matches forcing all three sub-agents:
 254 the best recorded operating points average 73.2% accuracy with 1.03 calls, against 73.6% for forced-
 255 all with 3.0 calls, a threefold reduction in sub-agent calls for a 0.4-point difference. The residual
 256 difference is concentrated on GPQA, where forced-all remains 6.0 points better; on MedQA the gap
 257 is 1.5 points. MMLU-Pro is the cleanest efficiency result: 1.07 selected calls exactly match three
 258 forced calls at 71.5%. AQuA-RAT shows that more calls can be harmful: 1.03 selected calls reach
 259 81.1%, whereas forced-all reaches 75.2%.

260 **Rescue and corruption under the learned policy.** Accuracy alone hides two opposing effects of
 261 delegation. Table 8 decomposes the selected high-call operating points into the unconditional proba-
 262 bility mass of wrong candidate solutions that become correct (*correction*) and correct candidates that
 263 become wrong (*corruption*). Every policy has positive net value, but the balance differs substantially.
 264 AQuA-RAT corrects 48.0% of all examples while corrupting 2.4%, a twenty-to-one ratio; GPQA
 265 corrects 19.0% while corrupting 4.0%. This explains why a high oracle does not by itself justify

Table 5: Development-set operating points with the same manager and sub-agent pool. These are diagnostic, validation-selected results rather than additional held-out tests. Accuracy values are percentages.

Task	Selected MARGENT		Force all		Δ Acc.
	Acc.	Calls	Acc.	Calls	
MedQA	85.0	1.000	86.5	3.0	-1.5
MMLU-Pro	71.5	1.065	71.5	3.0	0.0
GPQA	55.0	1.040	61.0	3.0	-6.0
AQuA-RAT	81.1	1.031	75.2	3.0	+5.9
Macro mean	73.2	1.034	73.6	3.0	-0.4

calling all sub-agents: an oracle retains only successful branches, whereas an executable policy must incur its own mistakes.

Does the policy call on the right states? For analysis only, we use gold labels to compute the *call gap*, the difference between the probability of delegating when the candidate solution is wrong and when it is correct. At the selected development operating points, this gap is positive on all four tasks: 0.381 on MedQA, 0.296 on MMLU-Pro, 0.067 on GPQA, and 0.102 on AQuA-RAT. The small GPQA gap is consistent with its weaker policy result: when candidate solutions are hard to distinguish, the manager approaches indiscriminate calling. A positive gap alone does not prove sub-agent-specific marginal-value prediction. A stricter matched-budget replay fixes the role and call count and resamples examples 2,000 times: MedQA’s selected subset reaches 82.0%, above the random 95% interval [75.0, 81.0], while MMLU-Pro (59.5% vs. [54.0, 60.5]) and GPQA (51.0% vs. [44.0, 53.0]) remain inside their intervals. The strongest better-than-random selection evidence is therefore on MedQA; this test does not isolate candidate use from query difficulty.

Commit examples control the call-stop trade-off. The commit-to-rescue cap ρ is the only changed data-construction variable within each pair in Table 9. Retaining more commit examples lowers the call rate in every pair. The call gap generally increases at the same time, so the remaining calls become more concentrated on wrong candidate solutions; accuracy, however, is not monotone. In particular, the aggressive AQuA-RAT comparison removes 37.4% of calls but also loses 12.6 points. Thus ρ is a transparent control over policy behavior, not a free efficiency improvement.

Across representative operating points, call gap is smallest on GPQA (0.017) and AQuA-RAT (0.152) and larger on MMLU-Pro (0.225) and MedQA (0.291), a descriptive pattern suggesting that selective calling is harder when candidate solutions are near chance.

4.5 Does the candidate solution matter to a delegated call?

The central mechanism predicts that a sub-agent’s value can depend on the solution it receives, not only on the question. On AQuA-RAT, the same ratio-zero checkpoint delegates on every example; blanking only the `current_solution` argument to the Verifier, without retraining or changing the call budget, lowers accuracy from 81.1% to 74.0%. Correction falls from 0.480 to 0.409 while corruption is essentially unchanged (0.0240 vs. 0.0236). Thus the candidate is a useful audit target for delegated computation. Because routing is fixed in this run, the intervention does not by itself show that the router uses the candidate to decide whether to call.

Outcome-only GRPO continuations move to 2.98 and 3.00 calls per example, while three additional MedQA seeds give 80.5 ± 1.0 accuracy at 0.517 ± 0.043 calls; Appendix D records these diagnostics.

5 Conclusion

The experiments show a simple but important result: the benefit of another agent depends on what it changes from the current reasoning state. MARGENT makes this comparison observable and turns it into supervision, recovering most measured oracle gain with few calls. In our setting, additional intelligence is most useful when allocated by measured value rather than invoked by default.

References

- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025. doi: 10.48550/arXiv.2503.13657.
- Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. ChatEval: Towards better LLM-based evaluators through multi-agent debate. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=FQepisCUWu>.
- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023. doi: 10.48550/arXiv.2305.05176.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11733–11763. PMLR, 2024. URL <https://proceedings.mlr.press/v235/du24e.html>.
- Sugyeong Eo, Hyeonseok Moon, Evelyn Hayoon Zi, Chanjun Park, and Heuiseok Lim. Debate only when necessary: Adaptive multiagent collaboration for efficient LLM reasoning. *arXiv preprint arXiv:2504.05047*, 2025. doi: 10.48550/arXiv.2504.05047.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021. doi: 10.3390/app11146421.
- Zhuofeng Li, Haoxiang Zhang, Seungju Han, Sheng Liu, Jianwen Xie, Yu Zhang, Yejin Choi, James Zou, and Pan Lu. In-the-flow agentic system optimization for effective planning and tool use. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://agentflow.stanford.edu/>.
- Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. Program induction by rationale generation: Learning to solve and explain algebraic word problems. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 158–167. Association for Computational Linguistics, 2017. doi: 10.18653/v1/P17-1015. URL <https://aclanthology.org/P17-1015/>.
- Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. A dynamic LLM-powered agent network for task-oriented agent collaboration. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=XII0Wp1XA9>.
- Sumeet Ramesh Motwani, Chandler Smith, Rocktim Jyoti Das, Rafael Rafailov, Ivan Laptev, Philip H. S. Torr, Fabio Pizzati, Ronald Clark, and Christian Schroeder de Witt. MALT: Improving reasoning with multi-agent LLM training. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=jXP9bgFack>.
- Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7076–7087. PMLR, 2020. URL <https://proceedings.mlr.press/v119/mozannar20b.html>.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024. doi: 10.48550/arXiv.2406.18665.

- Chanwoo Park, Seungju Han, Xingzhi Guo, Asuman E. Ozdaglar, Kaiqing Zhang, and Joo-Kyung Kim. MAPoRL: Multi-agent post-co-training for collaborative large language models with reinforcement learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30215–30248. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1459. URL <https://aclanthology.org/2025.acl-long.1459/>.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- Nenad Tomašev, Matija Franklin, and Simon Osindero. Intelligent AI delegation. *arXiv preprint arXiv:2602.11865*, 2026. doi: 10.48550/arXiv.2602.11865. URL <https://arxiv.org/abs/2602.11865>.
- Rajeev Verma, Daniel Barrejon, and Eric Nalisnick. Learning to defer to multiple experts: Consistent surrogate losses, confidence calibration, and conformal ensembles. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 11415–11434. PMLR, 2023. URL <https://proceedings.mlr.press/v206/verma23a.html>.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-3018. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ad236edc564f3e3156e1b2feafb99a24-Abstract.html. Datasets and Benchmarks Track.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>.
- Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. MasRouter: Learning to route LLMs for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.757. URL <https://aclanthology.org/2025.acl-long.757/>.
- Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=LkzuPorQ5L>.

A Additional experimental details

- Selection metadata.** The locked checkpoints are MedQA sft_3.0_s1, AQuA-RAT sft_1.0, MMLU-Pro sft_2.0, and GPQA sft_2.0. MedQA uses the seed closest to the mean of the replicated configuration rather than the best seed. The GPQA evaluation set is GPQA-Diamond-100 and is disjoint from the 446-example collection pool.
- Depth accounting.** The oracle rows in Table 1 are derived from the deepest primary collection for each task: depth three for MedQA and AQuA-RAT, and depth two for MMLU-Pro and GPQA. The “best measured” column is therefore a measured upper bound within the enumerated action sequences, not an upper bound over arbitrary prompts, models, or tools. At these depths, no measured sequence rescues 31/400 MedQA, 51/400 AQuA-RAT, 106/400 MMLU-Pro, and 61/200 GPQA root states; as specified in Section 3.3, these roots do not generate routing labels. A separate GPQA depth-three repeat used a different root collection and is not mixed into the primary depth ladder.

B Predecessor scaling diagnostic

For three of the four final tasks, before the marginal-value experiments, we evaluated cold-start, generic, and task-specific multi-agent training variants with Qwen3-4B, Qwen3-8B, and Qwen3.5-9B backbones. The development sets contained 100–200 examples per task. Table 6 reports the best result among the variants at each size minus the corresponding same-size base model. The mean gain shrinks sharply at 9B, and the 9B GPQA gain is zero. Because the maximum is selected across multiple variants and the sample sizes are small, this is evidence for the motivating tension rather than a scaling law or a main comparison.

Table 6: Retrospective gain of the best predecessor multi-agent variant over its same-size base model (percentage points).

Task	Qwen3-4B	Qwen3-8B	Qwen3.5-9B
MedQA	+16.5	+20.0	+4.0
GPQA	+13.0	+6.0	+0.0
MMLU-Pro	+30.5	+22.5	+10.5
Macro gain	+20.0	+16.2	+4.8

The experimental lineage also clarifies why the present method uses supervised marginal-value distillation. Outcome-only GRPO and additional sub-agent SFT did not reliably improve the earlier pipeline, whereas manager-side cold-start SFT improved most task–size cells. These preliminary variants differed in more than one factor and are not used as ablations in the main paper; their role was to motivate the controlled same-state collection.

C Selectivity diagnostics

Table 7: Candidate accuracy, random-guess margin, and call gap at representative development operating points. Margin is candidate accuracy minus $1/K$ for a K -choice task.

Task	Choices K	Candidate	Margin	Call gap
MedQA	4	0.685	0.435	0.291
MMLU-Pro	10	0.560	0.460	0.225
AQuA-RAT	5	0.390	0.190	0.152
GPQA	4	0.420	0.170	0.017

The table does not identify a causal relationship between candidate accuracy and selectivity. It records a practical failure boundary: on GPQA, where the candidate is only 17 points above chance, the policy’s call gap is numerically close to indiscriminate calling and the available sample is too small to resolve a small difference reliably.

Additional development diagnostics. The following tables support the main-text correction/corruption and commit-to-rescue analyses.

Table 8: Outcome decomposition at the recorded high-call MARGENT operating points. Correction and corruption are unconditional fractions of all examples, not conditional rates among called examples.

Task	Calls	Correction	Corruption
MedQA	1.000	0.210	0.035
MMLU-Pro	1.065	0.250	0.045
GPQA	1.110	0.190	0.040
AQuA-RAT	1.031	0.480	0.024

Table 9: Controlled commit-to-rescue comparisons on development data. “All” retains all available commit examples. Each arrow changes only ρ within a fixed collection and training recipe.

Task / collection	ρ	Accuracy	Calls	Call gap
MedQA depth 2	0 $\rightarrow$ 3	85.0 $\rightarrow$ 82.0	1.000 $\rightarrow$ 0.405	.000 $\rightarrow$.474
MedQA depth 1	1 $\rightarrow$ 2	80.5 $\rightarrow$ 82.5	.700 $\rightarrow$.560	.299 $\rightarrow$.381
MMLU-Pro depth 1	1 $\rightarrow$ 2	64.0 $\rightarrow$ 65.0	.475 $\rightarrow$.390	.289 $\rightarrow$.296
AQuA-RAT depth 3	0 $\rightarrow$ all	81.1 $\rightarrow$ 68.5	1.000 $\rightarrow$.626	.000 $\rightarrow$.195

Table 10: Recorded GRPO continuations using terminal binary correctness and no call penalty. Calls are mean sub-agent calls per example.

Run	Calls	Accuracy	Observed behavior
MedQA, 8B manager	2.98	–	approaches all three calls
MedQA, 9B manager	3.00	–	all three calls; solution emitted late

D Outcome-only reinforcement-learning continuations

Both continuations begin from selective SFT checkpoints and converge to the three-call ceiling. Accuracy records for these two runs are missing from the exported experiment record and are not imputed; this appendix therefore supports a claim about call behavior under a terminal-correctness reward, not about the accuracy cost of the collapse.

For one fixed MedQA SFT configuration, three additional training seeds yield 80.5 ± 1.0 accuracy and 0.517 ± 0.043 calls (mean $\pm$ standard deviation); the remaining task-level comparisons are single runs.

E Reproducibility scope

All main numerical claims can be reconstructed from the per-example branch and evaluation records used to export the accompanying dashboard. The workshop manuscript intentionally does not report token or latency comparisons because those measurements were not collected. A complete release should additionally include optimizer settings, hardware, prompts, LoRA configuration, stable example IDs for every split, and per-example outputs required for paired confidence intervals.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the problem, the state-conditioned delegation setting, the marginal-value measurement, and the scope of the reported empirical claims.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The manuscript states scope boundaries alongside the relevant experiments and in the reproducibility appendix, including the fixed three-agent pool, mostly single-run comparisons, and the use of sub-agent calls as the available resource measure.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present theoretical theorems or formal proof claims; its equations define measured quantities and the training procedure.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: The manuscript specifies the algorithm, data splits, checkpoint selection, and main evaluation protocol, but the exported record does not retain every synthesis prompt, optimizer setting, hardware field, or stable example identifier needed for full reproduction.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The anonymized supplementary artifact provides the implementation, generated branch and evaluation records, environment and execution instructions, and scripts for reproducing the reported tables.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [No]

Justification: Section 4 reports benchmark splits, model roles, decoding, search depths, selection procedure, baselines, and metrics, but some optimizer, LoRA, prompt, and hardware metadata are absent from the exported record.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We report mean and standard deviation over three additional seeds for one MedQA configuration in Section 4.5; the other main task results are single runs, so comprehensive error bars or significance tests are not available.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The paper reports model size and mean sub-agent calls, but the exported experimental record does not contain complete GPU, memory, runtime, or total-compute metadata.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research uses public research benchmarks and models, does not involve human subjects or private data, and was conducted in accordance with the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The current manuscript focuses on technical resource allocation and does not include a separate analysis of both positive and negative societal impacts.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release a new general-purpose pretrained model, scraped dataset, or other asset identified as posing a high risk of misuse.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All models and datasets are credited through their original papers. The final experiments use Qwen3.5-9B together with MedQA, MMLU-Pro, GPQA, and AQuA-RAT; the submission materials record the corresponding source and license information where available.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The anonymized supplementary artifact documents the released implementation and generated branch and evaluation records, including data provenance, file structure, configuration, and reproduction commands.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The study does not use crowdsourcing or conduct research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The study does not involve human subjects, so IRB approval or an equivalent review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

798 Question: Does the paper describe the usage of LLMs if it is an important, original, or
799 non-standard component of the core methods in this research? Note that if the LLM is used
800 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
801 scientific rigor, or originality of the research, declaration is not required.

802 Answer: [\[Yes\]](#)

803 Justification: Sections 3.1–3.3 and 4 describe the LLM manager and frozen sub-agents, their
804 roles, interaction protocol, decoding, and policy-training procedure.

805 Guidelines:

- 806 • The answer [\[N/A\]](#) means that the core method development in this research does not
807 involve LLMs as any important, original, or non-standard components.
- 808 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
809 be described.